

Exercícios 105.1 - Respostas

=====

1. Apenas um arquivo de inicialização é usado, de acordo com a seguinte ordem de precedência:

```
~/.bash_profile
~/.bash_login
~/.profile
```

2. (a) Com aspas duplas, o conteúdo de var2 é mostrado. Isso acontece porque, com aspas simples, a substituição de \$var1 e \$var2 pelos valores é feita no sub-shell, enquanto que, com aspas duplas, a substituição é feita antes da invocação de 'bash -c'. Em outras palavras, a linha de comando executada é

```
$ bash -c "echo bagre frito"
```

- (b) Existem dois casos a considerar. O primeiro é quando a variável é modificada depois de ter sido exportada e antes do sub-shell ser criado, e o segundo é quando a variável é modificada depois do sub-shell já ter sido criado. Para o primeiro caso, a modificação é refletida automaticamente, como mostra a sequência de comandos abaixo:

```
$ var1=baiacu
$ export var1
$ var1=bagre
$ bash -c 'echo $var1'
bagre
```

No segundo caso, a mudança não é refletida automaticamente em um sub-shell, como mostra a sequência de comandos abaixo:

```
$ bash -i
$ echo PID $$: var1=$var1
PID 12347: var1=bagre
$ kill -STOP $$ ### suspende o sub-shell
```

```
[1]+ Parado bash -i
$ var1=linguado ### modifica a variável
$ echo $var1
linguado
$ fg %1 ### retoma o sub-shell
bash -i
$ echo PID $$: var1=$var1
PID 12347: var1=bagre
```

- (c) Quando uma variável exportada é modificada em um sub-shell, essa mudança não é refletida no shell principal.
- (d) Caso uma variável seja modificada após a atribuição, isso não tem reflexo nas variáveis definidas usando a que foi modificada:

```
$ var3="ensopado de $var1"
$ echo $var3
ensopado de bagre
$ var1=linguado
$ echo $var3
ensopado de bagre
```

O uso de export não faz nenhuma diferença.

3. Os nomes de variáveis são sensíveis a caso, como mostra a sequência de comandos abaixo:

```
$ var1=bagre
$ Var1=baiacu
$ VAR1=badejo
```

```
$ echo $var1 $Var1 $VAR1
bagre baiacu badejo
```

4. unset remove a definição tanto de variáveis do shell (não exportadas) quanto de ambiente (exportadas). A remoção de uma variável de ambiente em um shell não afeta essa variável nos demais shells em que ela já estiver definida.

5. declare +x zz1

6. (a) 0 cp copia o arquivo azul (\$arq) para d1.

```
$ arq=azul
$ cp $arq d1
$ ls -l d1
total 4
-rw-rw-r-- 1 dcc dcc 5 set 22 14:40 azul
```

- (b) Mudando arq para 'baleia azul', os arquivos 'baleia' e 'azul' são copiados para d1. Isso ocorre porque, após a substituição de arq, o comando executado é "cp baleia azul d1" (cp recebe 3 argumentos, 2 arquivos e um diretório)

```
$ arq='baleia azul'
$ cp $arq d1
$ ls -l d1
total 8
-rw-rw-r-- 1 dcc dcc 5 set 22 14:40 azul
-rw-rw-r-- 1 dcc dcc 7 set 22 14:40 baleia
```

7. Para fazer o que o exercício pede, a solução trivial não funciona:

```
$ fruta=banana
$ doce=$frutada ; echo $doce
(em branco, a variável $frutada não existe)
```

É necessário saber onde termina o nome da variável e onde começa o sufixo. Isso pode ser feito de duas formas, delimitando a variável (primeiro caso abaixo) ou usando aspas simples ou duplas para delimitar o sufixo (segundo e terceiro casos):

```
$ doce=${fruta}da ; echo $doce
bananada
$ doce=$fruta"da" ; echo $doce
bananada
$ doce=$fruta'da' ; echo $doce
bananada
```

8. Os dois comandos geram saídas praticamente idênticas, a única diferença está na variável \$_, que representa o nome de caminho completo para o comando (/usr/bin/env ou /usr/bin/printenv).

9. (a) \$ echo \$USER
udesc
\$ echo \$PWD
/tmp

- (b) Alterar PWD muda o prompt do shell (PS1 usa \w), mas não altera o diretório corrente. Por outro lado, tem efeito sobre "cd -":

```
$ pwd
/tmp
$ PWD=/etc
$ cd - ; pwd
/home/udesc
$ cd - ; pwd
/etc
```

Alterar USER não tem efeito prático (embora o conteúdo da

variável seja mudado).

10. No diretório pai, o comando `whoami` executa o comando `/usr/bin/whoami`. No diretório `d1`, o script `whoami` é executado:

```
$ whoami
udesc
$ cd d1
$ whoami
root
```

Não se deve colocar o diretório corrente no `PATH` para evitar o risco de inadvertidamente executar programas com o mesmo nome de comandos do sistema.

11. (a) O alias já estava definido:

```
$ alias ls
alias ls='ls --color=auto'
```

(b) O novo alias mostra saída colorida.

```
$ alias lsi='ls -li'
```

(c) Ao remover o alias de `ls`, tanto `ls` quanto `lsi` deixam de mostrar saída colorida.

```
$ unalias ls
```

12. O alias tem precedência. Para usar a função, pode-se escapar o alias com `\`.

```
$ alias xpto='echo alias'
$ function xpto { echo funcao; }
$ xpto
alias
$ \xpto
funcao
```

- 13.

```
$ function peixe { echo manjuba; }      # (a)
$ declare -F
declare -f __expand_tilde_by_ref
(...)
declare -f peixe
(...)

$ declare -f peixe                      # (b)
peixe ()
{
    echo manjuba
}

$ bash -c peixe                        # (c) -- não funciona
bash: peixe: command not found

$ export -f peixe                      # (d) -- agora funciona
$ bash -c peixe
manjuba

$ peixe=merluza                        # (e) -- funciona

$ set                                  # (f)
(...)
peixe=merluza
(...)
peixe ()
{
```

```

    echo manjuba
}
(...)

$ env
(...)
peixe=merluza
(...)
BASH_FUNC_peixe%%=( ) { echo manjuba
}
(...)
```

(f) Tanto a função peixe quanto a variável peixe aparecem em set e env, mas de formas diferentes.

```

$ unset -f peixe                                # (g) -- não funciona
$ /bin/bash -c peixe
/bin/bash: peixe: command not found

$ peixe                                          # (h)
peixe: command not found
```

(h) No primeiro caso, o shell pai reporta o erro que foi detectado e informado pelo programa que ele executou ("/bin/bash:"), que é o sub-shell. No segundo caso, é o próprio shell pai que detecta e reporta o erro.

```

14. $ function param {
    echo "A funcao recebeu $# parametros"
    echo "Os parametros sao: $@"
}
```

15. Parâmetros contendo espaços são contados uma única vez, não sendo possível diferenciá-los na saída.

```

$ param a b "c d e" f
A funcao recebeu 4 parametros
Os parametros sao: a b c d e f
```

```

16. $ function aspas { egrep \".+\\" $@ ; }
```

```

17. $ function findc {
> find "$1" -name '*. [ch]' -print -o -name Makefile -print ;
> }
```